1 O que é encapsulamento?

Encapsular dados em uma classe: é juntar atributos e métodos na mesma classe e “esconder” os detalhes internos, evitando que qualquer parte do código mexa direto nos dados.
public: pode acessar de qualquer lugar.
protected: só a própria classe e as classes filhas conseguem acessar.
private: só a própria classe pode acessar.
Por que não deixar tudo public: porque qualquer parte do código poderia alterar os dados de forma errada, quebrando regras do sistema e causando bugs.

2 Para que servem os métodos get e set?

Por que usar getAtributo() ao invés de acessar direto: porque o método `get` mantém o controle da leitura dos dados. Se precisar mudar algo no futuro, alterar só o método e não o sistema inteiro.
O que o set faz além de atribuir valor: ele pode validar, corrigir, converter ou bloquear dados inválidos antes de salvar no atributo.
Exemplo prático:

```
public function setPreco($preco) {
        if ($preco > 0) {
            $this->preco = $preco;
         } else {
            echo "Erro: O preço deve ser maior que zero!<br>";
        }
}
```

3 Verdadeiro ou Falso — encapsulamento

(F) Um atributo private pode ser lido diretamente fora da classe.
Justificativa:
Atributos privados são estritamente inacessíveis fora do escopo da própria classe. Tentar lê-los externamente gera um erro fatal de execução (Fatal Error).

(F) Getters e setters só fazem sentido quando os atributos são private.
Justificativa:
Eles também são perfeitamente aplicáveis a atributos com visibilidade protected e mantêm o valor de centralizar regras de negócio e formatações de dados.

(V) Encapsulamento aumenta a segurança e a manutenção do código.
Justificativa:
Ele isola o estado interno contra interferências externas (segurança) e centraliza as regras de modificação em um único lugar (fácil manutenção).

(V) É possível ter um getter sem ter um setter para o mesmo atributo.
Justificativa:
Isto é ideal para criar atributos de "apenas leitura" (read-only),
como IDs de registros ou saldos consolidados que não devem
ser alterados de forma direta.

4	**Erro: acesso a atributo privado**

Linha com erro: echo $p->nome;
Explicação: O atributo $nome é restrito como private, impedindo o acesso direto fora do escopo da classe Produto.

```
<?php
class Produto {
    private $nome;
    private $preco;

    public function __construct($nome, $preco) {
        $this->nome = $nome;
        $this->preco = $preco;
    }

    // Getter para leitura pública
    public function getNome() {
        return $this->nome;
    }
}

$p = new Produto('Teclado', 150.00);

// Acesso indireto via método público
echo $p->getNome();
?>
```

5	**Erro: uso de variável sem $this**

Linha com erro: echo 'Aluno: ' . $nome . ' - Nota: ' . $nota;
Explicação: Em métodos de classe no PHP, a referência a atributos da própria instância exige imperativamente o uso da pseudo-variável
$this->. Escrever apenas $nome faz o PHP buscar uma variável local inexistente dentro daquele método.

```
<?php
class Aluno {
    private $nome;
    private $nota;

    public function __construct($nome, $nota) {
        $this->nome = $nome;
```

```
        $this->nota = $nota;
    }

    public function exibir() {
        echo 'Aluno: ' . $this->nome . ' - Nota: ' . $this->nota;
    }
}

$a = new Aluno('Carlos', 8.5);
$a->exibir();
?>
```

6	**Erro: setter não bloqueia valor inválido**

Trecho com erro: Lógica interna do método setIdade().
Explicação: O bloco if exibe a mensagem de erro, mas não interrompe a execução do método.
Assim, a atribuição $this->idade = $idade; é executada mesmo que o valor seja inválido.

```
<?php
class Pessoa {
    private $idade;

    public function setIdade($idade) {
        if ($idade < 0) {
            echo 'Idade invalida!<br>';
            return;
        }

        $this->idade = $idade;
    }

    public function getIdade() {
        return $this->idade;
    }
}

$p = new Pessoa();
$p->setIdade(-5);

echo $p->getIdade();
?>
```

7	**Erro: instanciação sem new**

Linha com erro: $c = Carro('Fusca');
Explicação: Em PHP, para criar/instanciar um objeto a partir de uma classe invocando o seu construtor, é obrigatória a utilização da palavra-chave new. Sem ela, o PHP interpreta a linha como uma chamada a uma função global inexistente.

```php
<?php
class Carro {
    private $modelo;

    public function __construct($modelo) {
        $this->modelo = $modelo;
    }

    public function getModelo() {
        return $this->modelo;
    }
}

$c = new Carro('Fusca');

echo $c->getModelo();
?>
```

8	**Classe Livro**

```php
<?php
class Livro {
    private $titulo;
    private $autor;
    private $paginas;

    public function __construct($titulo, $autor, $paginas) {
        $this->setTitulo($titulo);
        $this->setAutor($autor);
        $this->setPaginas($paginas);
    }

    public function setTitulo($titulo) {
        if (trim($titulo) !== "") {
            $this->titulo = trim($titulo);
        }
    }

    public function getTitulo() {
        return $this->titulo;
    }

    public function setAutor($autor) {
        if (trim($autor) !== "") {
            $this->autor = trim($autor);
        }
    }

    public function getAutor() {
        return $this->autor;
    }
```

```
    public function setPaginas($paginas) {
        if (is_numeric($paginas) && $paginas > 0) {
            $this->paginas = $paginas;
        }
    }

    public function getPaginas() {
        return $this->paginas;
    }

    public function exibir() {
        echo "Livro: " . $this->getTitulo() .
             " | Autor: " . $this->getAutor() .
             " | Páginas: " . $this->getPaginas() . "<br>";
    }
}

$livro1 = new Livro("Dom Casmurro", "Machado de Assis", 256);
$livro2 = new Livro("O Alquimista", "Paulo Coelho", 172);

$livro1->exibir();
$livro2->exibir();
?>
```

9	**Classe ContaBancaria**

```
<?php
class ContaBancaria {
    private $titular;
    private $saldo;

    public function __construct($titular) {
        $this->titular = $titular;
        $this->saldo = 0;
    }

    public function getTitular() {
        return $this->titular;
    }

    public function getSaldo() {
        return $this->saldo;
    }

    public function depositar($valor) {
        if ($valor > 0) {
            $this->saldo += $valor;
        }
    }
```

```
    public function sacar($valor) {
        if ($valor > 0 && $valor <= $this->saldo) {
            $this->saldo -= $valor;
        } else {
            echo "Saldo insuficiente!<br>";
        }
    }
}

$conta = new ContaBancaria("Ana Souza");

$conta->depositar(500);
$conta->sacar(200);

echo "Saldo final de " . $conta->getTitular() . ": R$ " . $conta->getSaldo() . "<br>";
?>
```

10 Classe Aluno com média

```
<?php
class Aluno {
    private $nome;
    private $nota1;
    private $nota2;

    public function __construct($nome, $nota1, $nota2) {
        $this->nome = $nome;
        $this->setNota1($nota1);
        $this->setNota2($nota2);
    }

    public function setNota1($nota) {
        if ($nota >= 0 && $nota <= 10) {
            $this->nota1 = $nota;
        }
    }

    public function getNota1() {
        return $this->nota1;
    }

    public function setNota2($nota) {
        if ($nota >= 0 && $nota <= 10) {
            $this->nota2 = $nota;
        }
    }

    public function getNota2() {
        return $this->nota2;
    }
```

```
    public function calcularMedia() {
        return ($this->nota1 + $this->nota2) / 2;
    }

    public function situacao() {
        $media = $this->calcularMedia();

        echo "Média: " . $media . " - " .
            ($media >= 5 ? 'Aprovado' : 'Reprovado') . "<br>";
    }
}

$aluno = new Aluno("Roberto", 6.5, 7.5);

$aluno->situacao();
?>
```

11 Classe Retangulo

```
<?php
class Retangulo {
    private $largura;
    private $altura;

    public function __construct($largura, $altura) {
        $this->setLargura($largura);
        $this->setAltura($altura);
    }

    public function setLargura($l) {
        if ($l > 0) {
            $this->largura = $l;
        }
    }

    public function getLargura() {
        return $this->largura;
    }

    public function setAltura($a) {
        if ($a > 0) {
            $this->altura = $a;
        }
    }

    public function getAltura() {
        return $this->altura;
    }

    public function calcularArea() {
        return $this->largura * $this->altura;
```

```
    }

    public function calcularPerimetro() {
        return 2 * ($this->largura + $this->altura);
    }
}

$r1 = new Retangulo(5, 10);
$r2 = new Retangulo(6, 8);

if ($r1->calcularArea() > $r2->calcularArea()) {
    echo "Retângulo 1 tem maior área.<br>";
} else {
    echo "Retângulo 2 tem maior área.<br>";
}
?>
```

12 Classe Funcionario com salário

```
<?php
class Funcionario {
    private $nome;
    private $cargo;
    private $salario;

    public function __construct($nome, $cargo, $salario) {
        $this->nome = $nome;
        $this->cargo = $cargo;
        $this->salario = $salario;
    }

    public function getSalario() {
        return $this->salario;
    }

    public function aumentarSalario($percentual) {
        $this->salario = $this->salario * (1 + $percentual / 100);
    }

    public function exibir() {
        echo "Nome: " . $this->nome .
            " | Salário: R$ " .
            number_format($this->salario, 2, ',', '.') . "<br>";
    }
}

$f = new Funcionario("Claudio Reis", "Supervisor", 3200.00);
$f->exibir();
$f->aumentarSalario(15);
$f->exibir();
?>
```

13 Classe Temperatura com conversão

```php
<?php
class Temperatura {
    private $celsius;

    public function __construct($celsius) {
        $this->celsius = $celsius;
    }

    public function setCelsius($c) {
        $this->celsius = $c;
    }

    public function getCelsius() {
        return $this->celsius;
    }

    public function paraFahrenheit() {
        return ($this->celsius * 9/5) + 32;
    }

    public function paraKelvin() {
        return $this->celsius + 273.15;
    }
}

$t = new Temperatura(37);

echo "C: " . $t->getCelsius() .
     " | F: " . $t->paraFahrenheit() .
     " | K: " . $t->paraKelvin() . "<br>";
?>
```

14 Classe Estoque com array de produtos

```php
<?php
class Estoque {
    private $produtos = array();

    public function adicionarProduto($nome, $quantidade) {
        $this->produtos[] = array(
            'nome' => $nome,
            'qtd' => $quantidade
        );
    }

    public function listarProdutos() {
        foreach ($this->produtos as $p) {
```

```
            echo "Produto: " . $p['nome'] .
                " | Qtd: " . $p['qtd'] . "<br>";
        }
    }

    public function totalItens() {
        $total = 0;

        foreach ($this->produtos as $p) {
            $total += $p['qtd'];
        }

        return $total;
    }
}

$e = new Estoque();

$e->adicionarProduto("Caderno", 50);
$e->adicionarProduto("Caneta", 120);

$e->listarProdutos();

echo "Total de itens: " . $e->totalItens() . "<br>";
?>
```

15 Classe Agenda de contatos (desafio)

```
<?php
class Contato {
    private $nome;
    private $telefone;

    public function __construct($nome, $telefone) {
        $this->nome = $nome;
        $this->telefone = $telefone;
    }

    public function getNome() {
        return $this->nome;
    }

    public function getTelefone() {
        return $this->telefone;
    }
}

class Agenda {
    private $contatos = array();

    public function adicionarContato($nome, $telefone) {
```

```
        $this->contatos[] = new Contato($nome, $telefone);
    }

    public function listar() {
        foreach ($this->contatos as $c) {
            echo "Nome: " . $c->getNome() .
                " | Telefone: " . $c->getTelefone() . "<br>";
        }
    }
}

$agenda = new Agenda();

$agenda->adicionarContato("Marcos Silva", "99999-1111");
$agenda->adicionarContato("Maria Clara", "98888-2222");

$agenda->listar();
?>
```